API-Based Products (S2-25_SEZG504)
ASSIGNMENT 1

NAME: SOUNDHARYA GP
ID: 2024TM93678

1. OpenAPI 3.1.1 – Record Label API (Q1)

This section describes the design of a RESTful API for a record label system using OpenAPI 3.1.1 specification. The API manages artist information including name, genre, number of albums published, and username. The API is secured using Basic Authentication to ensure only authorized users can access and modify data.

Features Implemented:

The API supports the following functionalities:

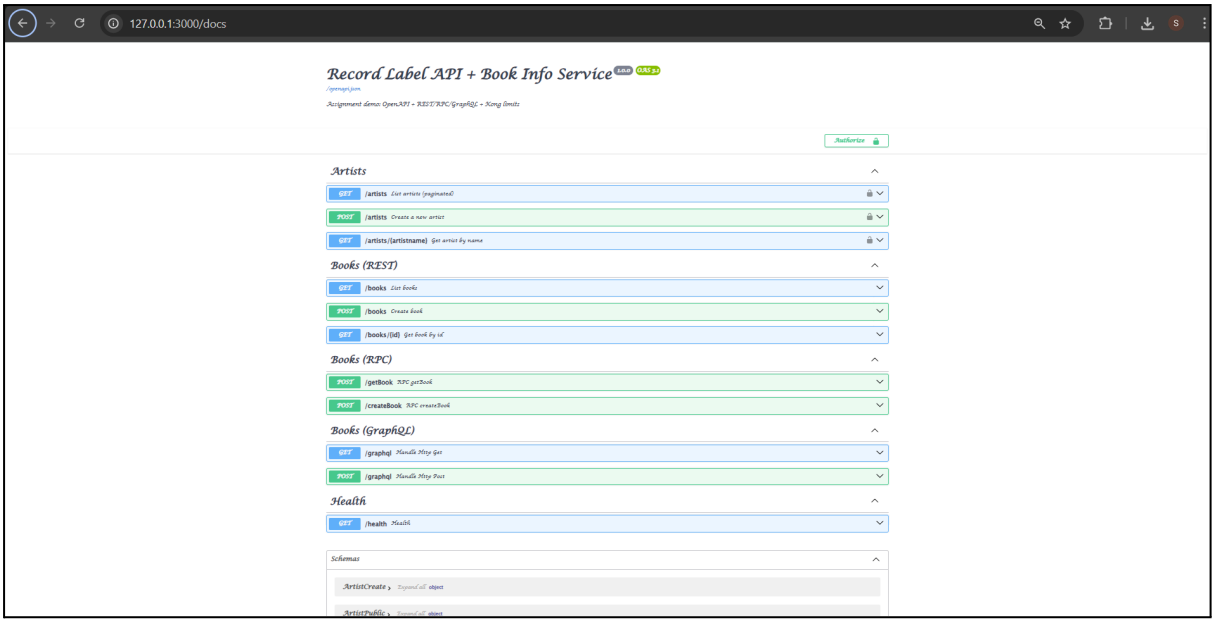
- Retrieve list of artists with pagination (offset, limit)
- Create a new artist entry
- Fetch artist details by artist name
- Secure access using Basic Authentication
- Proper HTTP status codes for success and failure cases

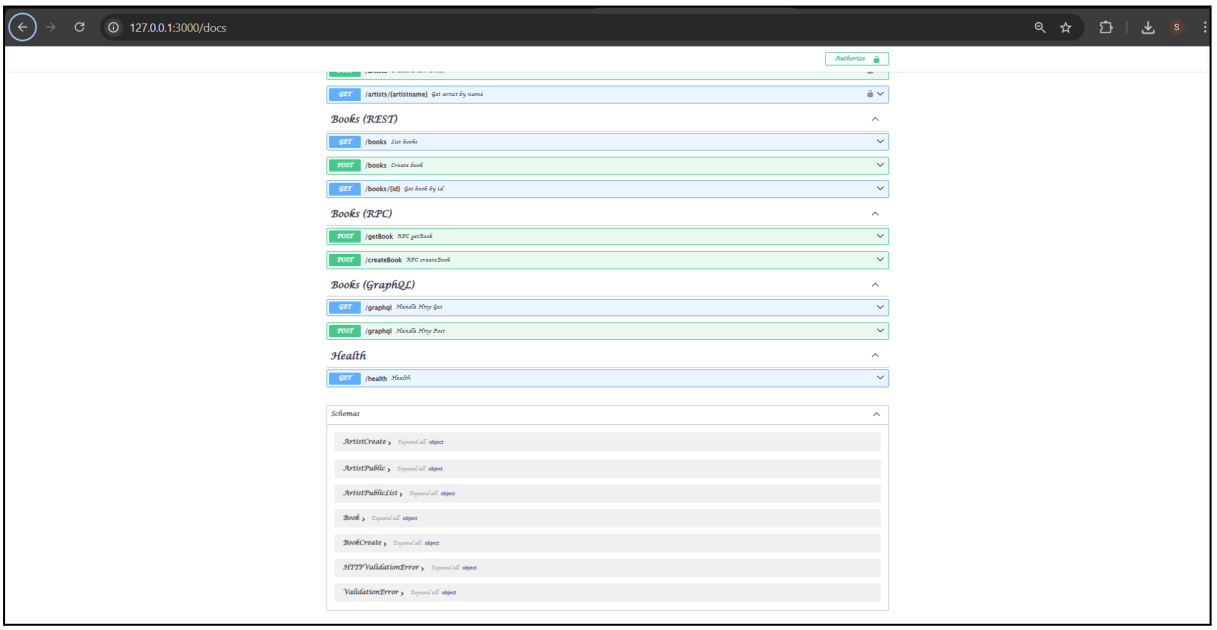
Endpoints Overview:

Method	Endpoint	Description
GET	/artists	List all artists (paginated)
POST	/artists	Create new artist
GET	/artists/{artistname}	Fetch artist by name

Security:
The API is protected using **Basic Authentication**:
- Username: admin
- Password: admin123
- Unauthorised access returns - **401** Unauthorized

Screen Shots:
1.Swagger UI





2.Unauthorized request (401)

```
PROBLEMS 4   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS        powershell - apiassignment1-sound

PS C:\Users\Soundharya GP\OneDrive\Desktop\API Project\Assignment_1> cd .\apiassignment1-sound\
PS C:\Users\Soundharya GP\OneDrive\Desktop\API Project\Assignment_1\apiassignment1-sound> cd .\apiassignment1-so
und\
PS C:\Users\Soundharya GP\OneDrive\Desktop\API Project\Assignment_1\apiassignment1-sound\apiassignment1-sound> c
url.exe -i http://localhost:3000/artists
HTTP/1.1 401 Unauthorized
date: Mon, 27 Apr 2026 13:34:54 GMT
server: uvicorn
www-authenticate: Basic
content-length: 25
content-type: application/json

{"detail":"Unauthorized"}
PS C:\Users\Soundharya GP\OneDrive\Desktop\API Project\Assignment_1\apiassignment1-sound\apiassignment1-sound>
```

3.Authorized GET with pagination (200)

```
PROBLEMS 4   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS        powershell - apiassignment1-sound

PS C:\Users\Soundharya GP\OneDrive\Desktop\API Project\Assignment_1\apiassignment1-sound\apiassignment1-sound> curl.exe -i -u admin:admin123 "http://loca
lhost:3000/artists?offset=0&limit=2"
HTTP/1.1 200 OK
date: Mon, 27 Apr 2026 13:39:23 GMT
server: uvicorn
content-length: 198
content-type: application/json

{"items":[{"name":"Radiohead","genre":"Alternative","albumsPublished":9,"username":"radiohead"},{"name":"Adele","genre":"Pop","albumsPublished":4,"userna
me":"adele"}],"offset":0,"limit":2,"total":3}
PS C:\Users\Soundharya GP\OneDrive\Desktop\API Project\Assignment_1\apiassignment1-sound\apiassignment1-sound>
```

4.Create artist (201) and User exists (400)

```
PROBLEMS 4   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS        powershell

PS C:\Users\Soundharya GP\OneDrive\Desktop\API Project\Assignment_1> curl.exe -i -u admin:admin123 -X POST "http://127.0.0.1:3000/artists" -H "Content-Ty
pe: application/json" -d '{\"name\":\"Ben\",\"genre\":\"Southern\",\"albumsPublished\":1,\"username\":\"ben\"}'
HTTP/1.1 400 Bad Request
date: Mon, 27 Apr 2026 16:43:24 GMT
server: uvicorn
content-length: 34
content-type: application/json

{"detail":"Artist already exists"}
PS C:\Users\Soundharya GP\OneDrive\Desktop\API Project\Assignment_1> curl.exe -i -u admin:admin123 -X POST "http://127.0.0.1:3000/artists" -H "Content-Ty
pe: application/json" -d '{\"name\":\"sam\",\"genre\":\"Southern\",\"albumsPublished\":6,\"username\":\"sam\"}'
HTTP/1.1 201 Created
date: Mon, 27 Apr 2026 16:44:22 GMT
server: uvicorn
content-length: 70
content-type: application/json

{"name":"sam","genre":"Southern","albumsPublished":6,"username":"sam"}
PS C:\Users\Soundharya GP\OneDrive\Desktop\API Project\Assignment_1>
```

5.Not found response (404 case)

```
PROBLEMS 4   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                    powershell - apiassignment1-sound

PS C:\Users\Soundharya GP\OneDrive\Desktop\API Project\Assignment_1\apiassignment1-sound\apiassignment1-sound> curl.exe -i -u admin:admin123 http://local
host:3000/artists/NoSuchArtist
HTTP/1.1 404 Not Found
date: Mon, 27 Apr 2026 13:43:21 GMT
server: uvicorn
content-length: 29
content-type: application/json

{"detail":"Artist not found"}
PS C:\Users\Soundharya GP\OneDrive\Desktop\API Project\Assignment_1\apiassignment1-sound\apiassignment1-sound>
```

Observation

The pagination system reduces payload size and improves performance by allowing clients to request data in controlled chunks.

2. Kong API Gateway – Rate Limiting & Security (Q2)

This section demonstrates the use of Kong API Gateway to enforce security policies such as rate limiting and request size limiting. Kong acts as a reverse proxy and controls traffic before it reaches the backend API.

Features Implemented

- Rate limiting per minute per client
- Request throttling using Kong plugins
- Request size validation
- Proxy-based request routing to backend service

Rate Limiting Behavior

Kong restricts the number of requests allowed per time window. Once the limit is exceeded, the API returns a limit-exceeded response and stops further processing.

Evidence from headers:

- X-RateLimit-Limit-Minute
- X-RateLimit-Remaining-Minute
- X-Kong-Proxy-Latency
- Via: kong/3.7.1

Request Size Limiting

Kong enforces payload size restrictions. If the request body exceeds allowed limits, it rejects the request with:

- HTTP 413 Payload Too Large

Screen Shots

- Successful Kong request (200 OK with rate limit headers)

```
PS C:\Users\Soundharya GP\OneDrive\Desktop\API Project\Assignment_1\apiassignment1-sound\apiassignment1-sound> curl.exe -i -u admin:admin123 "http://loca
lhost:8000/artists?offset=0&limit=2"
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 198
Connection: keep-alive
X-RateLimit-Remaining-Minute: 8
X-RateLimit-Limit-Minute: 10
RateLimit-Remaining: 8
RateLimit-Limit: 10
RateLimit-Reset: 36
date: Mon, 27 Apr 2026 15:57:23 GMT
server: uvicorn
X-Kong-Upstream-Latency: 8
X-Kong-Proxy-Latency: 105
Via: kong/3.7.1
X-Kong-Request-Id: ab922323b8e44f02baad1750fd4bb422

{"items":[{"name":"Radiohead","genre":"Alternative","albumsPublished":9,"username":"radiohead"},{"name":"Adele","genre":"Pop","albumsPublished":4,"userna
me":"adele"}],"offset":0,"limit":2,"total":3}
PS C:\Users\Soundharya GP\OneDrive\Desktop\API Project\Assignment_1\apiassignment1-sound\apiassignment1-sound>
```

- Rate limit exceeded response (loop test)

```
PROBLEMS 4   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   powershell - apiassignment1-sound

PS C:\Users\Soundharya GP\OneDrive\Desktop\API Project\Assignment_1\apiassignment1-sound\apiassignment1-sound> for ($i=1; $i -le 15; $i++) { curl.exe -s
-o $null -w "%{http_code}`n" -u admin:admin123 http://localhost:8000/artists }
tronic","albumsPublished":4,"username":"daftpunk"}],"offset":0,"limit":10,"total":3}{"items":[{"name":"Radiohead","genre":"Alternative","albumsPublished"
:9,"username":"radiohead"},{"name":"Adele","genre":"Pop","albumsPublished":4,"username":"adele"},{"name":"Daft Punk","genre":"Electronic","albumsPublishe
d":4,"username":"daftpunk"}],"offset":0,"limit":10,"total":3}{"items":[{"name":"Radiohead","genre":"Alternative","albumsPublished":9,"username":"radiohea
d"},{"name":"Adele","genre":"Pop","albumsPublished":4,"username":"adele"},{"name":"Daft Punk","genre":"Electronic","albumsPublished":4,"username":"daftpu
nk"}],"offset":0,"limit":10,"total":3}{"items":[{"name":"Radiohead","genre":"Alternative","albumsPublished":9,"username":"radiohead"},{"name":"Adele","ge
nre":"Pop","albumsPublished":4,"username":"adele"},{"name":"Daft Punk","genre":"Electronic","albumsPublished":4,"username":"daftpunk"}],"offset":0,"limit
":10,"total":3}{"items":[{"name":"Radiohead","genre":"Alternative","albumsPublished":9,"username":"radiohead"},{"name":"Adele","genre":"Pop","albumsPubli
shed":4,"username":"adele"},{"name":"Daft Punk","genre":"Electronic","albumsPublished":4,"username":"daftpunk"}],"offset":0,"limit":10,"total":3}{"items"
:[{"name":"Radiohead","genre":"Alternative","albumsPublished":9,"username":"radiohead"},{"name":"Adele","genre":"Pop","albumsPublished":4,"username":"ade
le"},{"name":"Daft Punk","genre":"Electronic","albumsPublished":4,"username":"daftpunk"}],"offset":0,"limit":10,"total":3}{"items":[{"name":"Radiohead","
genre":"Alternative","albumsPublished":9,"username":"radiohead"},{"name":"Adele","genre":"Pop","albumsPublished":4,"username":"adele"},{"name":"Daft Punk
","genre":"Electronic","albumsPublished":4,"username":"daftpunk"}],"offset":0,"limit":10,"total":3}{"items":[{"name":"Radiohead","genre":"Alternative","a
lbumsPublished":9,"username":"radiohead"},{"name":"Adele","genre":"Pop","albumsPublished":4,"username":"adele"},{"name":"Daft Punk","genre":"Electronic",
"albumsPublished":4,"username":"daftpunk"}],"offset":0,"limit":10,"total":3}{
  "message":"API rate limit exceeded",
  "request_id":"8c24986314d49057b0285d703d79716c"
}{
  "message":"API rate limit exceeded",
  "request_id":"4d97f304cfade144ce9b0f1962fe2c2a"
}{
  "message":"API rate limit exceeded",
  "request_id":"a3f9fa6abda1c420cc51b10e88763768"
}{
  "message":"API rate limit exceeded",
  "request_id":"59d4c45751d7dc672a6ab87173a52e18"
}{
  "message":"API rate limit exceeded",
  "request_id":"7f83b819d73a7132111e4f3f58bff203"
}
PS C:\Users\Soundharya GP\OneDrive\Desktop\API Project\Assignment_1\apiassignment1-sound\apiassignment1-sound>
```

- Request size limit violation (413 error)

```
PROBLEMS 4   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   powershell - apiassignment1-sound

PS C:\Users\Soundharya GP\OneDrive\Desktop\API Project\Assignment_1\apiassignment1-sound\apiassignment1-sound> python -c "import requests; big={'name':'X
'*200000,'genre':'Y','albumsPublished':1,'username':'u'}; r=requests.post('http://localhost:8000/artists',auth=('admin','admin123'),json=big); print(r.st
atus_code, r.text[:100])"
413 {
  "message":"Request size limit exceeded",
  "request_id":"821fc21d84a0f11ad059c61d7c867a03"
}
PS C:\Users\Soundharya GP\OneDrive\Desktop\API Project\Assignment_1\apiassignment1-sound\apiassignment1-sound>
```

Observation

Kong successfully enforces gateway-level security without modifying backend code, demonstrating separation of concerns in API architecture.

3. Book Info Service – REST, RPC, GraphQL (Q3)

This section demonstrates three different API paradigms implemented for the same dataset (Book Information Service). The goal is to compare REST, RPC, and GraphQL approaches.

Dataset

Each book contains:

- ID
- Title
- Author

Example: “1”- ”Clean Code” - “Robert C. Martin”

REST API

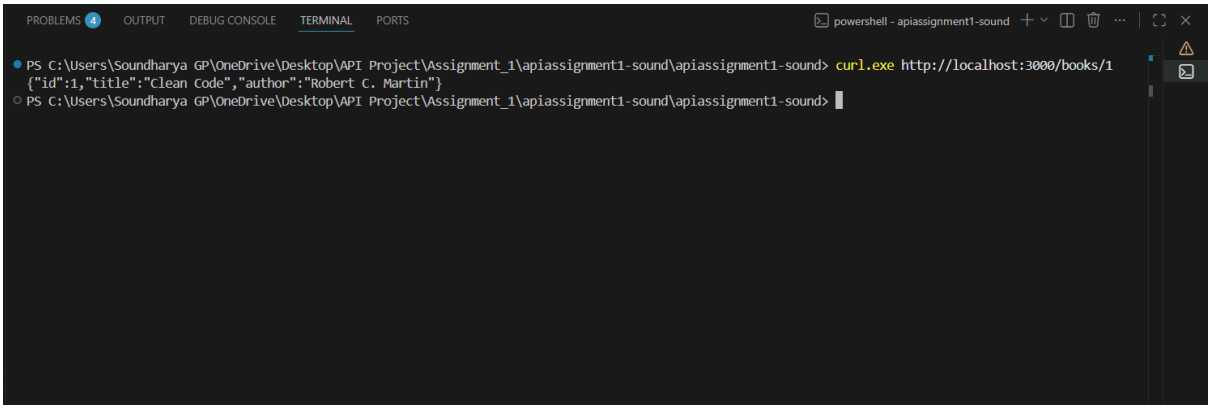
Endpoint: GET /books/{id}

Description:

REST follows a resource-based approach where each book is treated as a resource. The client fetches data using standard HTTP methods.

Response:

Returns a full book object in JSON format.



RPC API

Endpoint: POST /getBook

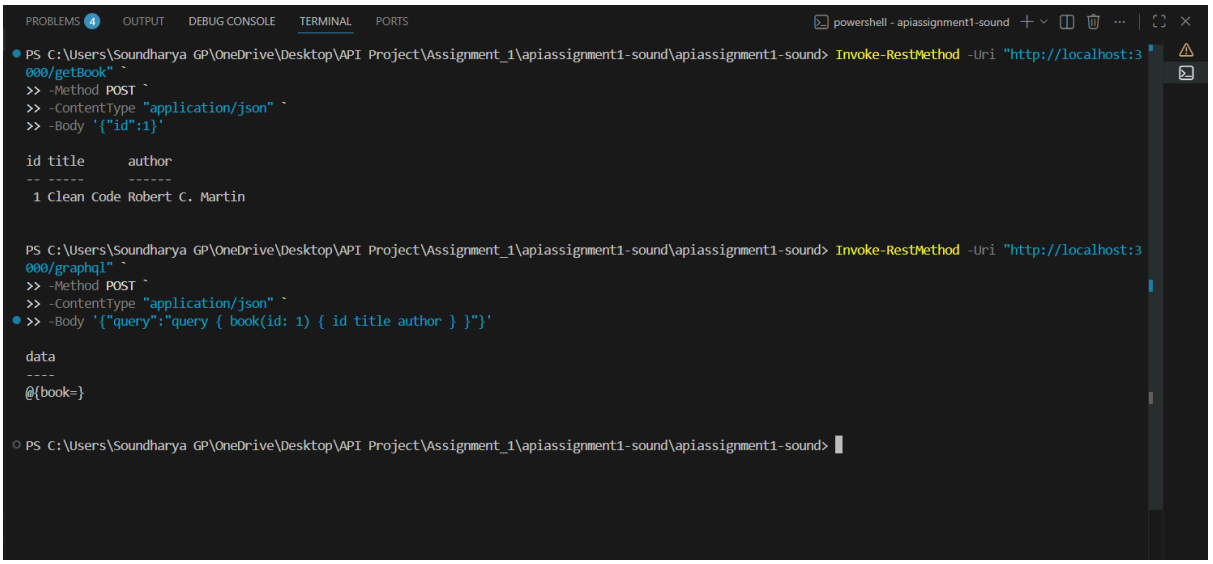
Description:

RPC follows an action-based approach where the client calls a function-like endpoint with input parameters.

Input: { "id": 1 }

Response:

Returns the requested book details.



GraphQL API

Endpoint: POST /graphql

Description:

GraphQL allows clients to request exactly the data they need using flexible queries.

Query Example:

```
query {
    book(id: 1) {
        id title author
        }
}
```

Response:

Returns structured JSON with only requested fields.

```
PROBLEMS 4   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                    powershell - apiassignment1-sound

PS C:\Users\Soundharya GP\OneDrive\Desktop\API Project\Assignment_1\apiassignment1-sound\apiassignment1-sound> Invoke-RestMethod -Uri "http://localhost:3
000/graphql" `
>> -Method POST `
>> -ContentType "application/json" `
>> -Body '{"query":"query { book(id: 1) { id title author } }"}' | ConvertTo-Json -Depth 5
{
    "data":  {
                 "book":  {
                              "id":  1,
                              "title":  "Clean Code",
                              "author":  "Robert C. Martin"
                          }
             }
}
PS C:\Users\Soundharya GP\OneDrive\Desktop\API Project\Assignment_1\apiassignment1-sound\apiassignment1-sound>
```

For book id=1, **REST-** */books/1*, **RPC-** */getBook*, and **GraphQL-** *book(id:1)* return the same title and author values.

Comparison of API Paradigms:

Feature	REST	RPC	GraphQL
Style	Resource-based	Action-based	Query-based
Flexibility	Medium	Low	High
Overfetching	Yes	Yes	No
Use Case	Standard APIs	Internal calls	Flexible frontend APIs

All three paradigms return consistent data but differ in structure and flexibility. GraphQL provides the most efficient data retrieval, while REST is the most widely used standard.

Demonstrated practical implementation of API design using OpenAPI standards, API security using Kong Gateway, and multiple API paradigms (REST, RPC, GraphQL). The system ensures secure, scalable, and flexible data access patterns suitable for modern distributed applications.